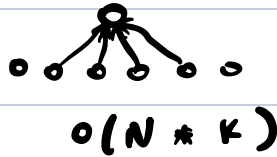
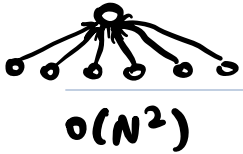


HYENA MODELS

Attn.

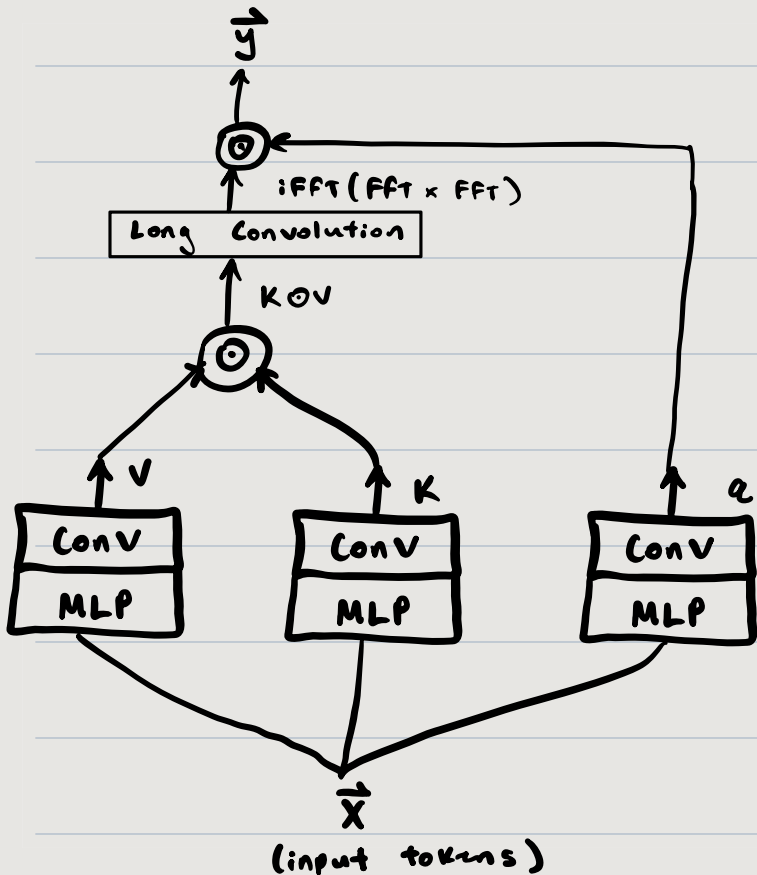
vs.

Convolutions



} can we get best of both worlds?

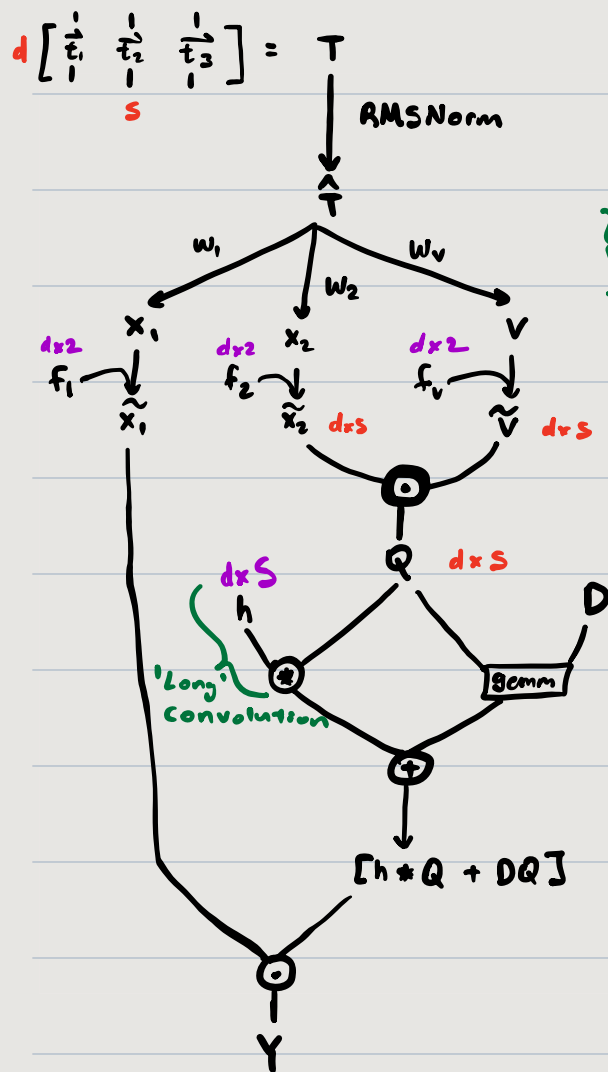
Hyena primitive



```
class Hyena(nn.Module):
    def __init__(self, d_model, seq_len, short_kernel=3):
        self.norm1 = RMSNorm(d_model)
        self.qkv = nn.Linear(d_model, d_model * 3)
        self.short_conv = nn.Conv1d(3 * d_model, 3 * d_model,
                                     kernel_size=short_kernel)
        self.h = generate_long_conv_filter

    def long_conv(self, q):
        Q = FFT(q)
        H = FFT(self.h)
        c = iFFT(Q * H)
        return c
```

Full Compute graph



Dense
Conv

gate # 1

Long Convolution

gate # 2

Short Convolution

$$\begin{matrix} \vec{t}_1 & \vec{t}_2 & \vec{t}_3 & \vec{t}_4 \end{matrix}$$



$$\begin{bmatrix} t_{11} & t_{21} & t_{31} & t_{41} \\ t_{12} & t_{22} & t_{32} & t_{42} \\ t_{13} & t_{23} & t_{33} & t_{43} \end{bmatrix}$$

$$\begin{bmatrix} t_{11} & t_{21} & t_{31} & t_{41} \\ t_{12} & t_{22} & t_{32} & t_{42} \\ t_{13} & t_{23} & t_{33} & t_{43} \end{bmatrix}$$

$$\begin{bmatrix} t_{11} & t_{21} & t_{31} & t_{41} \\ t_{12} & t_{22} & t_{32} & t_{42} \\ t_{13} & t_{23} & t_{33} & t_{43} \end{bmatrix}$$

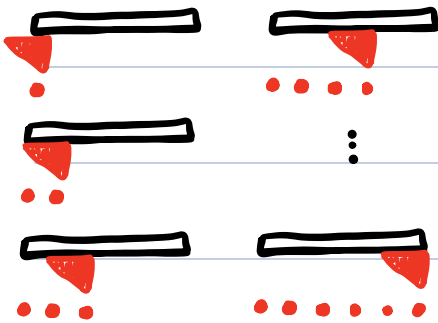
$$\begin{bmatrix} w_a & w_d \\ w_b & w_c \\ w_e & w_f \end{bmatrix}$$

$$\vec{w}_a \vec{w}_d$$

$$Y = \begin{bmatrix} w_d t_{11} & \dots \\ w_e t_{12} & \dots \\ w_f t_{13} & \dots \end{bmatrix}$$

$$\begin{bmatrix} w_a & w_d \\ w_b & w_c \\ w_e & w_f \end{bmatrix}$$

$$Y = \begin{bmatrix} w_d t_{11} & w_a t_{11} + w_d t_{21} & \dots \\ w_e t_{12} & w_b t_{12} + w_c t_{22} & \dots \\ w_f t_{13} & w_e t_{13} + w_f t_{23} & \dots \end{bmatrix}$$



} Mixing along sequence dimension!

$$\vec{Y}[t] = \vec{w}[0] \odot \vec{T}[t] + \vec{w}[1] \odot \vec{T}[t-1]$$

$$\text{OR } \vec{w}[0] \odot \vec{T}[t] + \vec{w}[1] \odot \vec{T}[t-1] + \vec{w}[2] \odot \vec{T}[t-2]$$

$$\text{OR } \sum_{k=0}^{\text{Kern. Size}} \vec{w}[k] \odot \vec{T}[t-k]$$

$$= \vec{w}[0] \odot \vec{T}[t] + \vec{w}[1] \odot \vec{T}[t-1]$$

Convolution Equation

given input $T = \begin{bmatrix} \vec{T[0]} \\ | \\ \vec{T[1]} \\ | \\ \vdots \end{bmatrix}$ & kernel $W = \begin{bmatrix} \vec{W[0]} \\ | \\ \vdots \end{bmatrix}$

imagine FLIPPING W & sliding window:

$$W = \begin{bmatrix} \cdots & \vec{W[0]} \\ | \\ \vdots \end{bmatrix}$$

$$T = \begin{bmatrix} \vec{T[0]} & \vec{T[1]} & \vec{T[2]} & \vec{T[3]} & \cdots \end{bmatrix}$$

$Y =$



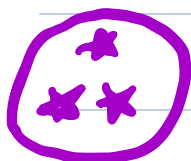
$$W = \begin{bmatrix} \cdots & \vec{W[0]} \\ | \\ \vdots \end{bmatrix}$$

$$T = \begin{bmatrix} \vec{T[0]} & \vec{T[1]} & \vec{T[2]} & \vec{T[3]} & \cdots \end{bmatrix}$$

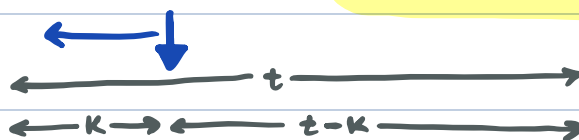
$Y =$



Ergo, $\vec{Y[t]} = \sum_{k=0}^{K-1} \vec{h[k]} \odot \vec{T[t-k]}$



$k=0$



filter starts at zero & increases leftward

input starts at t & decreases leftward

$$W = \begin{bmatrix} \cdots & \vec{W[0]} \\ | \\ \vdots \end{bmatrix}$$

$$T = \begin{bmatrix} \vec{T[0]} & \vec{T[1]} & \vec{T[2]} & \vec{T[3]} & \cdots \end{bmatrix}$$

Sidenote: Signal processing convolutions

- convs are an old-school signal processing tool!

↳ Continuous signals (e.g. voltage, sound, etc.) stand in contrast to discrete tokens!

$$\vec{y}(t) = \int_0^{+\infty} \underbrace{\vec{h}(\tau)}_{\substack{\text{Contrib. weight of} \\ \text{that past timestep}}} \odot \underbrace{\vec{x}(t-\tau)}_{\substack{\text{how 'far back' to look:} \\ \tau \text{ time units ago!}}} d\tau$$

Causal: only past contributes

↑
Current timestep

note y, h, x are signals: fns wrt time!

We say $\vec{y}(t)$ (evaluated at timestep t) is vector-valued

Thus, this is really:
$$\vec{y}(t) = \begin{bmatrix} \int h_0(\tau) x_0(t-\tau) d\tau \\ \int h_1(\tau) x_1(t-\tau) d\tau \\ \vdots \\ \int h_n(\tau) x_n(t-\tau) d\tau \end{bmatrix}$$

 $\underbrace{\hspace{1cm}}_{\vec{h}(\tau)} \quad \underbrace{\hspace{1cm}}_{\vec{x}(\tau)}$

This conv $\left(\int_{-\infty}^{+\infty} h(\tau) \cdot x(t-\tau) d\tau \right)$ is an LTI system

- Linear $S[ax_1 + bx_2] = aS[x_1] + bS[x_2]$

- Time invariant

if $x \xrightarrow{S} y$
then $x-t_0 \xrightarrow{S} y-t_0$

↑
fn $\rightarrow$ fn
mapping

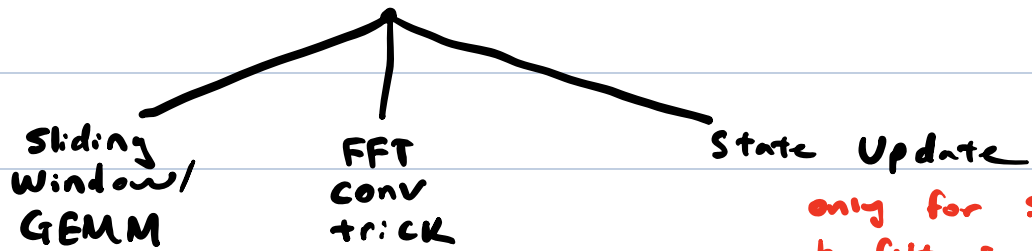
The integrals too hairy to compute analytically, so we discretize:

$$\int_a^b f(x) dx = \sum_{k=a}^b f(k) \cdot \Delta x$$

~~Algebra~~

$$y(t) = \int_0^{\infty} h(\tau) \cdot x(t-\tau) d\tau \approx \sum_{k=0}^{\infty} \underbrace{h(k \cdot \Delta)}_{\substack{k \\ \Delta \text{ steps}}} \cdot \underbrace{x(t-k \cdot \Delta)}_{\substack{t-k \Delta \\ \Delta \text{ steps}}} \cdot \underbrace{\Delta}_{\substack{\text{output system's} \\ \text{value}}} \cdot \underbrace{\Delta}_{\substack{\text{time step}}} \cdot \underbrace{\Delta}_{\substack{\text{Riemann} \\ \text{Rectangle}}}$$

Once you have a discrete conv sum:



only for special h filters, s.t.

$$\hat{y} = w_i \odot (h * z + D \odot z)$$

MLP filter input

$$\begin{aligned} \vec{V}_1 &= \text{FFT}(h) \\ \vec{V}_2 &= \text{FFT}(\hat{z}) \\ R_1 &= \vec{V}_1 \odot \vec{V}_2 \\ R_2 &= \text{iFFT}(R_1) \end{aligned}$$

• get kernel weights via:

tokens $\rightarrow$ MLP $\rightarrow$ weights



Can use tensor cores via flashFFTconv

$$S[n] = \alpha \cdot S[n-1] + \beta \cdot x[n]$$

$$y[n] = \gamma \cdot S[n]$$



Expands to conv. as per SSMS. SSMS have special filter

$$h[k] = C \cdot A^k \cdot B$$

$$O(K \cdot S)$$

$$O(S \log S)$$

$$O(1) \text{ wrt sequence length}$$